

User Flow

Assessment AI – User Flow

Assesment

User sees Categories.

Examples:

- Commerce
- Technology
- Healthcare

Under each category, Areas of Interest are displayed.

Examples:

Commerce

- Marketing
- Finance
- HR

Technology

- Software Development
- Data Analytics
- Cyber Security

User selects an Area of Interest.

Selection based continuation

The system checks the type of Area of Interest selected.

Option 1 – General Assessment

Examples:

- Aptitude
- Logical Reasoning
- Quantitative Ability
- Pattern Recognition

Since these are not linked to a specific job role, the user skips Job Title selection and goes directly to Assessment Setup.

Option 2 – Role-Based Assessment

Examples:

- Marketing
- Finance
- HR
- Software Development

The user proceeds to Job Title Selection.

Flow 1 – General Assessment

Once area of interest is selected

User selects:

- Difficulty
- Number of Questions
- Preview On/Off

User clicks **Generate Assessment**.

System loads questions from the Question Library. If all questions from library have been answered previously, it AI generates.

If Preview is ON:

- Show all questions
- Allow Replace Question
- Start Test

If Preview is OFF:

- Start Test immediately

After completion:

- Show Score
-

If non general assessment is chosen

User sees all Job Titles under the selected Area of Interest.

Example:

Area of interest - Marketing

They see job titles like -

- Social Media Manager
- Content Writer
- SEO Specialist

User selects a Job Title.

System checks which assessment type is assigned to that Job Title.

There is also a button:

Create Your Own Assessment

If this clicked, then they get the same screen as flow three. More information is given in flow three.

Flow 2 – Basic MCQ

User selects:

- Difficulty
- Experience Level
- Number of Questions
- Preview On/Off

They can also create their own assessment, as per flow 3.

User clicks Continue.

JD Screen

If JD is available, it automatically appears on screen.

User can:

Choose Existing JD

Choose a JD from the library.

Generate JD

Generate a JD using AI.

Paste JD

Paste their own JD.

User finalises the JD.

Skills Being Tested

Once a JD is selected, generated, or pasted, the system extracts and displays the skills that will be tested.

Example:

Skills Being Tested

- SEO
- Content Writing
- Social Media Marketing
- Analytics

The user can review these skills before generating the assessment. They can't edit.

Question Generation

The system first searches the Question Library using:

- Skill Tags
- Difficulty Level

Questions can be retrieved regardless of whether the JD came from:

- JD Library
- AI Generated JD

The JD source should not affect the ability to use library questions.

If enough questions are available:

- Load questions from the Question Library

If enough questions are not available:

- Generate the remaining questions using AI

Example:

User requests 20 questions.

Library provides 15 questions.

System generates 5 additional questions using AI.

Additional requirement - System should track which library questions the users has previously correctly answered. For a period of 30 days, the same questions should not be asked. No requirement to track AI questions, those will only be stored in temporary memory.

All AI questions generated should have a skill tracker attached, in order to reflect skill based percentage in the final report.

Preview Screen

If Preview is ON:

- Show all questions
- Allow Replace Question

No answers should be shown.

User clicks Start Test.

If Preview is OFF:

- Start Test immediately.
-

Assessment Screen

User completes the assessment.

After completion:

- Show Final Score
 - Show Final Percentage
 - Show skill based score
-

Flow 3 – Basic + Skills MCQ

Same flow as Basic MCQ.

Additional field:

Skills To Focus On

Examples:

- SEO
- Recruitment
- Google Ads
- Excel

Multiple skills can be selected.

Skills Being Tested

The system combines:

JD Skills

Example:

- SEO
- Analytics
- Social Media Marketing

User Selected Skills

Example:

- Google Ads
- Content Writing

Final Skills Being Tested:

- SEO
- Analytics
- Social Media Marketing
- Google Ads
- Content Writing

These skills are displayed before the assessment is generated.

If the system already has the skill logged in it will prompt. If not not they can not add their own random skills.

Question Generation

Priority order:

1. Selected Skills
2. JD Skills
3. Job Title Tags

If sufficient questions are not available as the library does not contain them or the user has already answered all questions correctly in last 30 days:

- Generate remaining questions using AI

Everything else remains the same as Basic MCQ.

Flow 4 – Coding Assessment Option

User selects:

- Difficulty
- Experience Level
- Number of Questions
- Skills To Focus On
- Assessment Mode
 - MCQ
 - Coding
- Preview On/Off

User clicks Continue.

JD Screen

Same as Basic + Skills MCQ.

User can:

- Select a JD from the library
 - Generate a JD using AI
 - Paste their own JD
-

Skills Being Tested

The system displays the final skills being tested based on:

- JD Skills
- User Selected Skills

Example:

- React

- JavaScript
- NodeJS
- SQL
- MongoDB

The user can review these skills before generating the assessment.

Assessment Generation

If MCQ Selected

Follow the Basic + Skills MCQ flow.

Questions are generated based on:

- Selected Skills
- JD Skills
- Job Title
- Difficulty

If Coding Selected

Coding challenges are generated based on:

- Selected Skills
- JD Skill tags
- Job Title tags

The system first searches the Coding Question Library.

If sufficient coding challenges are available:

- Use Coding Library Questions

If sufficient coding challenges are not available:

- Generate remaining coding challenges using AI

No additional setup screens are required.

Preview Screen

If Preview is ON:

Display:

- Coding Challenge Title
- Problem Statement

Allow Replace Question.

No solutions should be shown.

Coding Assessment

Display:

- Problem Statement
- Code Editor
- Run Code
- Submit Code

System evaluates submissions against predefined test cases.

Question Replacement Rules

Applicable to all assessment types.

When a user clicks Replace Question:

Priority Order:

1. Questions never attempted
2. Questions previously answered incorrectly
3. AI generated replacement

Questions previously answered correctly should not be reused whenever possible.

Difficulty Levels

All assessments support:

- Easy
- Medium
- Hard

Questions should be filtered based on the selected difficulty level.

Assessment Completion

For Version 1, the Results Screen only displays:

- Final Score
- Final Percentage
- Skill based score

The system should store:

- Questions answered correctly
- Questions answered incorrectly

No detailed analytics, skill breakdowns, AI feedback, or performance reports are required in Version 1.

Admin Dashboard

Admin Dashboard

Admin sees:

Categories & Areas of Interest

Example:

Commerce

- Marketing
- Finance
- HR

Technology

- Software Development
 - Data Analytics
-

Question Library

Separate menu item.

This shows all questions in the system.

Categories & Areas Of Interest

Admin can:

Add Category

Example:

- Commerce
 - Technology
-

Add Area Of Interest

Example:

Under Commerce:

- Marketing
- Finance
- HR

Under Technology:

- Software Development
 - Data Analytics
-

Select Area Of Interest

Example:

Technology

→ Software Development

Only now should Job Titles become visible.

Job Titles

Once an Area Of Interest is selected, display all Job Titles under it.

Example:

Technology

→ Software Development

Job Titles:

- Frontend Developer
 - Backend Developer
 - Full Stack Developer
-

Add Job Title

Admin enters:

- Job Title Name
- Assessment Type
 - Basic MCQ
 - Basic + Skills MCQ
 - Coding

Save.

Select Job Title

Example:

Full Stack Developer

Once selected, the admin can manage:

- Job Descriptions
 - Skills
 - Questions
-

Job Descriptions

Display all JDs under the selected Job Title.

Example:

Full Stack Developer

0-2 Years

- JD 1
- JD 2

2-5 Years

- JD 3
 - JD 4
-

Add JD

Options:

Manual Entry

Admin enters:

- Experience Band
 - JD
 - Skill tags (Either automate or they can also manually enter)
-

AI Generate

Admin enters:

- Job Title
- Experience Band

System generates:

- JD
 - Skills
-

CSV Upload

Upload multiple JDs.

System extracts skills automatically.

Skills

Under every JD display:

Skills Being Tested

Example:

- React
- NodeJS
- SQL
- API Development

Skills can be:

- Added manually
 - Generated automatically from AI
-

Questions Under Job Title

Inside the Job Title screen display:

Questions

This displays all questions linked to:

- The Job Title
 - Skills under that Job Title
-

Add Questions

Manual

Admin enters:

- Question
 - Answer
 - Difficulty
 - Skill Tags
 - Job Title Tags
-

AI Generate

Admin enters:

- Number of Questions
- Difficulty
- Skills

System generates:

- Questions
 - Answers
 - Tags
-

CSV Upload

Upload questions in bulk.

Go Live

New content is created as:

- Draft

Admin Level 2 can click:

Go Live

Only then does it become available to users.

Question Library

Separate section accessible from Dashboard.

This is the master question database.

Displays all questions in the system.

Filters:

- Skill
- Job Title
- Difficulty
- Assessment Type
- Draft / Live

Actions:

- View
- Edit (Admin 2)
- Delete (Admin 2)
- Go Live (Admin 2)

Database

Table count (what you actually need)

#	Table	Purpose
1	Category	Top group
2	AreaOfInterest	Area under category
3	JobTitle	Role (job-track only)
4	JobDescription	JD text
5	Skill	Master skill list
6	Question	MCQ + Coding
7	Admin	Admin users (optional now)
—	Link tables	—
8	JobDescriptionSkill	JD ↔ Skill
9	QuestionJobTitle	Question ↔ JobTitle
10	QuestionArea	Question ↔ Area
11	QuestionSkill	Question ↔ Skill

That's the whole admin DB.

Enums (fixed choices)

Name	Values	Meaning
CategoryKind	JOB_TRACK, TOPIC_TRACK	Does this category use job titles or only areas?
PublishStatus	DRAFT, LIVE	Hidden vs visible to users
AssessmentType	BASIC_MCQ, BASIC_SKILLS_MCQ, CODING	Test style
QuestionKind	MCQ, CODING	Question format
Difficulty	EASY, MEDIUM, HARD	Hardness

JdSource	MANUAL, AI, CSV	How JD was added
AdminRole	ADMIN_ADD_ONLY, ADMIN_FULL	Create vs Go Live

Tables explained

1. Category

Examples: Technology, Commerce, Aptitude

Field	Meaning
id	Primary key
name	Unique name
kind	JOB_TRACK or TOPIC_TRACK
status	DRAFT / LIVE

- JOB_TRACK → Area → JobTitle → JD → Questions
- TOPIC_TRACK → Area → Questions (skip JobTitle)

2. AreaOfInterest

Examples: Software Development · Marketing · Quant · Communication

Field	Meaning
id	Primary key
categoryId	Parent category
name	Unique within that category
status	DRAFT / LIVE
assessmentType	Optional — useful for TOPIC_TRACK (often always BASIC_MCQ)

3. JobTitle (*JOB_TRACK only*)

Examples: Frontend Developer, HR Executive

Field	Meaning
id	Primary key
areaId	Parent area
name	Unique within that area
assessmentType	Required — Basic MCQ / Basic+Skills / Coding
status	DRAFT / LIVE

4. JobDescription (*JOB_TRACK* only)

Field	Meaning
id	Primary key
jobTitleId	Parent job title
experienceBand	e.g. 0-2 Years
content	Full JD text
source	MANUAL / AI / CSV
status	DRAFT / LIVE

Skills attach via JobDescriptionSkill.

5. Skill

Shared list: React, SQL, Communication...

Field	Meaning
id	Primary key
name	Unique

Used by JDs and questions.

6. Question (one bank for everyone)

Field	MCQ	Coding
-------	-----	--------

prompt	Question text	Problem statement
kind	MCQ	CODING
difficulty	Yes	Yes
assessmentType	Yes	Yes
status	DRAFT/LIVE	DRAFT/LIVE
options	{ choices: [4], correctIndex }	null
answer	Correct option text	Expected output
testCases	null	{ shown: [], hidden: [] }
codingStarter	null	Optional

7–10. Link tables (why they exist)

Link	Connects	When admin uses it
JobDescriptionSkill	JD ↔ Skill	Skills on a JD
QuestionJobTitle	Question ↔ JobTitle	“Add by job title” (Technology/Commerce)
QuestionArea	Question ↔ Area	“Add by area” (Aptitude → Quant)
QuestionSkill	Question ↔ Skill	“Add by skill” (React coding, etc.)

Rule: every question must have at least one of: JobTitle, Area, or Skill.

11. Admin

Field	Meaning
id	Primary key
email	Login identity
name	Display
role	ADD_ONLY or FULL (Go Live)
isActive	On/off

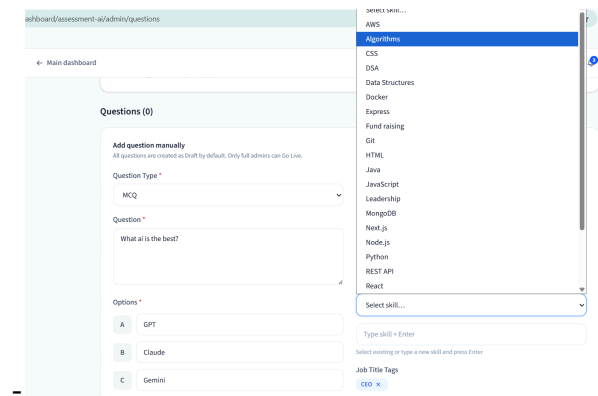
How questions get attached (examples)

Example	Category kind	Links
Frontend MCQ	JOB_TRACK	Question → JobTitle “Frontend”
React coding	JOB_TRACK	Question → Skill “React” (± JobTitle)
Quant aptitude	TOPIC_TRACK	Question → Area “Quant”
Communication MCQ	TOPIC_TRACK	Question → Area “Communication”

Tab 4

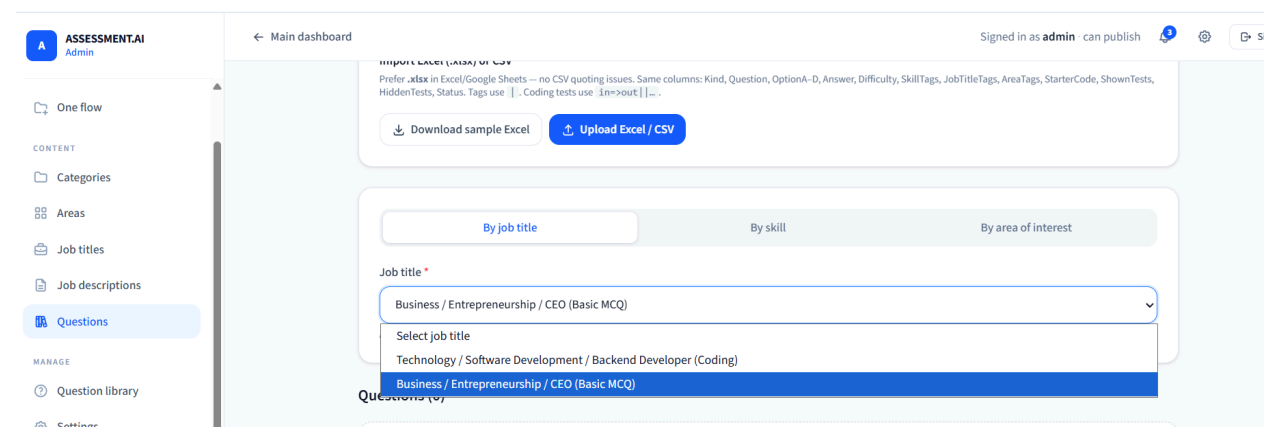
FIXES

- add + button on the right top corner
- Add filter on draft and live basis
- Add filter on the skill basis on the question tab
- one flow can start from any content
- Add an edit option for all the fields



If the admin has already entered the skill tags in the JD step then the skills added should appear right at the top under select skills before any other skills. They should be slightly highlighted as well (subtle).

- A search bar in the skills dropdown as well
- If the admin has already entered a job description and has clicked on the add question button, the admin should not be made to select the JD again in the dropdown. They can simply go back to the immediate previous step and make the correction if needed.

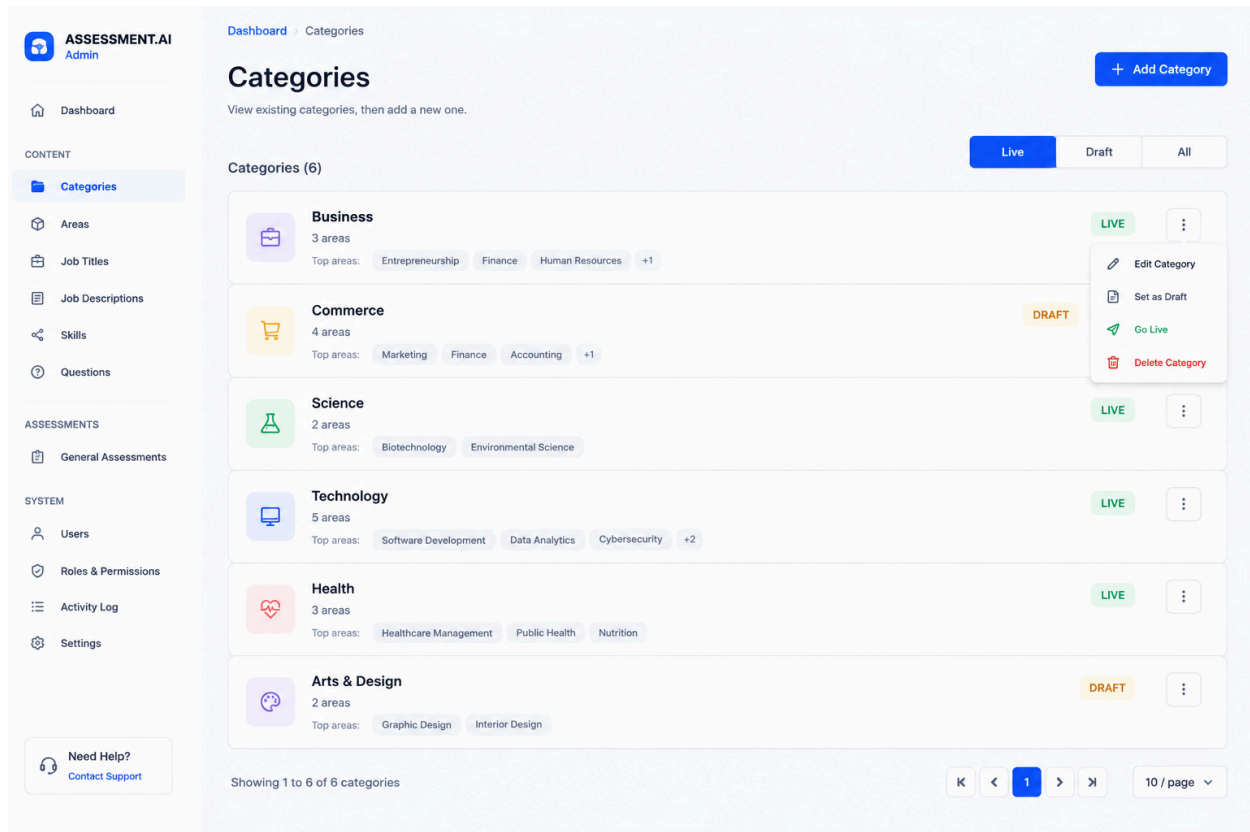


Search bar in the Job title including every other dropdown.

- Enable multiple selection in the dropdowns.

changes

catagory



Category Tab Changes

1. Add Category button

- The current button only shows a “+” icon.
- Change it to a full button that says “+ Add Category”.

2. Category status toggle

- Keep the three existing options:
 - Live
 - Draft
 - All
- Live must always be selected by default whenever the Category tab/page is opened.
- Users should still be able to manually switch between Live, Draft, and All.

3. Category row / preview

- Keep everything else in the existing design unchanged.
- Add a small visual indication showing some of the Areas of Interest contained within each category.
- This should give the admin a quick idea of what Areas of Interest belong to that category without having to open it.

4. Category row interaction

- Make the entire category row clickable, not just the edit button.

- Clicking anywhere on the category row should open/enable the category editing view.
- The existing Edit action should still be available, but the whole row should behave as the main clickable target.

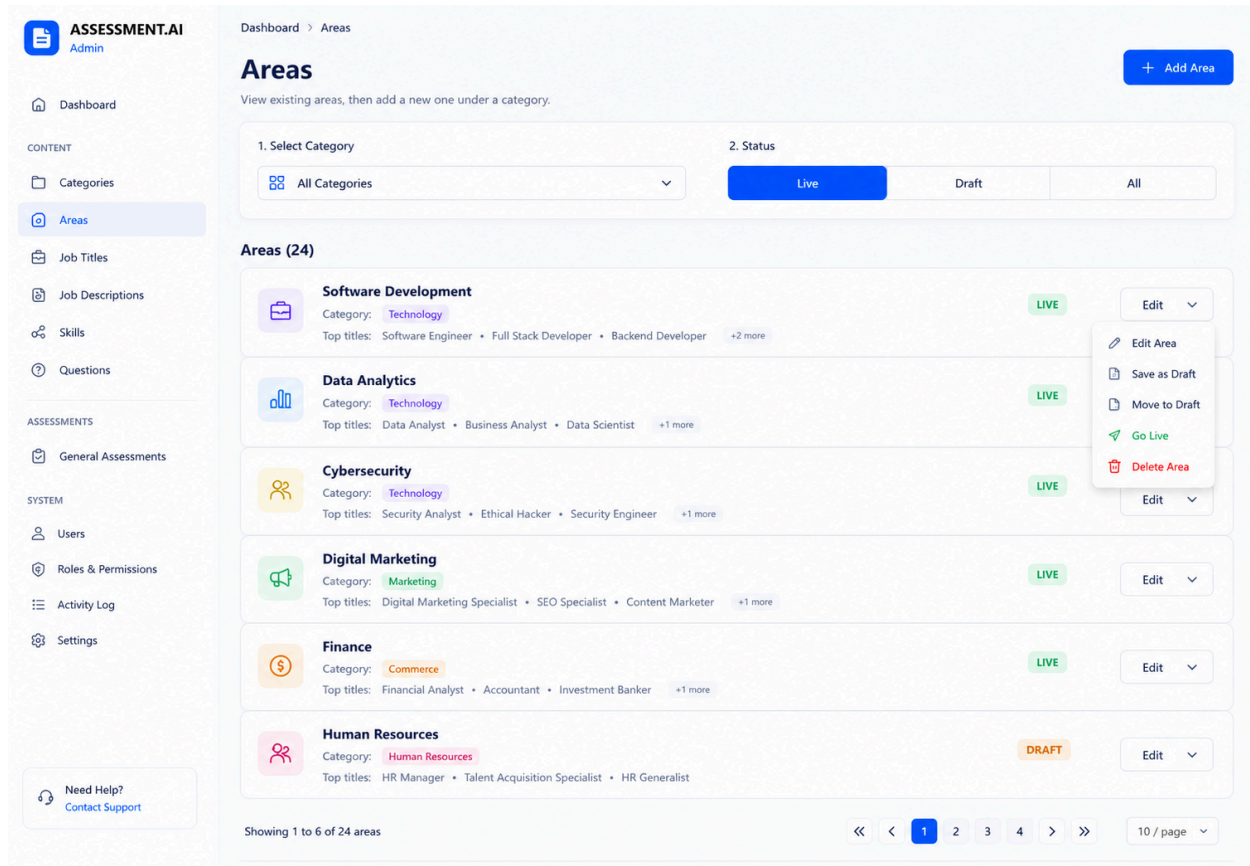
5. Edit dropdown

- Change the current Edit button into a dropdown/action menu.
- The dropdown should contain relevant category actions, such as:
 - Edit
 - Draft / Move to Draft
 - Go Live
 - Delete
 - Any other appropriate category management actions already supported by the system.
- The actions shown should depend on the category's current status where appropriate. For example, a Live category should not show “Go Live” as an active action if it is already live.

Important

Do not change the existing visual design unnecessarily. Keep the current layout, styling, spacing, typography, and overall UI exactly as it is. These are functional/UX improvements to the existing Category tab, not a redesign.

AREAS TAB



Category Tab — Required Changes

Please make these changes to the existing Category tab **without changing the current design unnecessarily**. Keep the existing UI, styling, spacing, typography, and overall visual language.

1. Category Filter

- Categories and Areas need to be properly separated.
- Add a **Category selection/filter dropdown** so the user can choose which category they want to view.
- The dropdown must include:
 - **All**
 - Individual categories
- **All** should show areas from every category.
- When a specific category is selected, show **only the areas belonging to that category**.

2. Status Filter

Keep the existing status toggle:

Live | Draft | All

- **Live must always be selected by default** whenever the page is opened/refreshed.
- Users can manually switch to **Draft** or **All**.
- The **Category filter and Status filter must work together**.

For example:

- **Category: Technology + Status: Live**
→ Show only Live Areas under Technology.
- **Category: All + Status: Live**
→ Show all Live Areas across every category.
- **Category: Technology + Status: All**
→ Show both Live and Draft Areas under Technology.

3. Add Area Button

- The current “+” **icon-only button** should be replaced.
- Make it a proper button displaying:

+ Add Area

4. Save as Draft

- Move the “**Save as Draft**” action into the **Edit/action dropdown**.
- Do not keep it as a separate prominent action if it currently appears elsewhere.
- The edit dropdown should contain the relevant actions for the Area, such as:
 - Edit
 - Save as Draft / Move to Draft
 - Go Live
 - Delete
 - Other relevant actions where applicable.
- The available actions should make sense based on the Area's current status.

5. Make Category Information More Visible

The **category associated with each Area needs to be much more obvious** in the Area listing.

When looking at an Area, the user should be able to immediately understand:

Area → Category → Top Titles/Careers

For example:

Software & Technology

Technology

Top titles: **Software Engineer · Full Stack Developer · AI Engineer**

The category should have clear visual hierarchy and should not feel like hidden metadata.

6. Show the Top Titles/Careers

For each Area, display the **top titles/careers associated with that Area** directly in the listing/card/row.

The user should be able to quickly understand what that Area contains without opening it.

Only show the most relevant/top titles in the main view so the UI does not become overcrowded. If there are more titles than can reasonably fit, use an appropriate “**+X more**” treatment or similar interaction consistent with the existing design.

Expected Final Structure

At the top of the page:

Category: All ▼

Status: Live | Draft | All
Live selected by default

Then display the Areas matching **both filters**.

Each Area should clearly communicate:

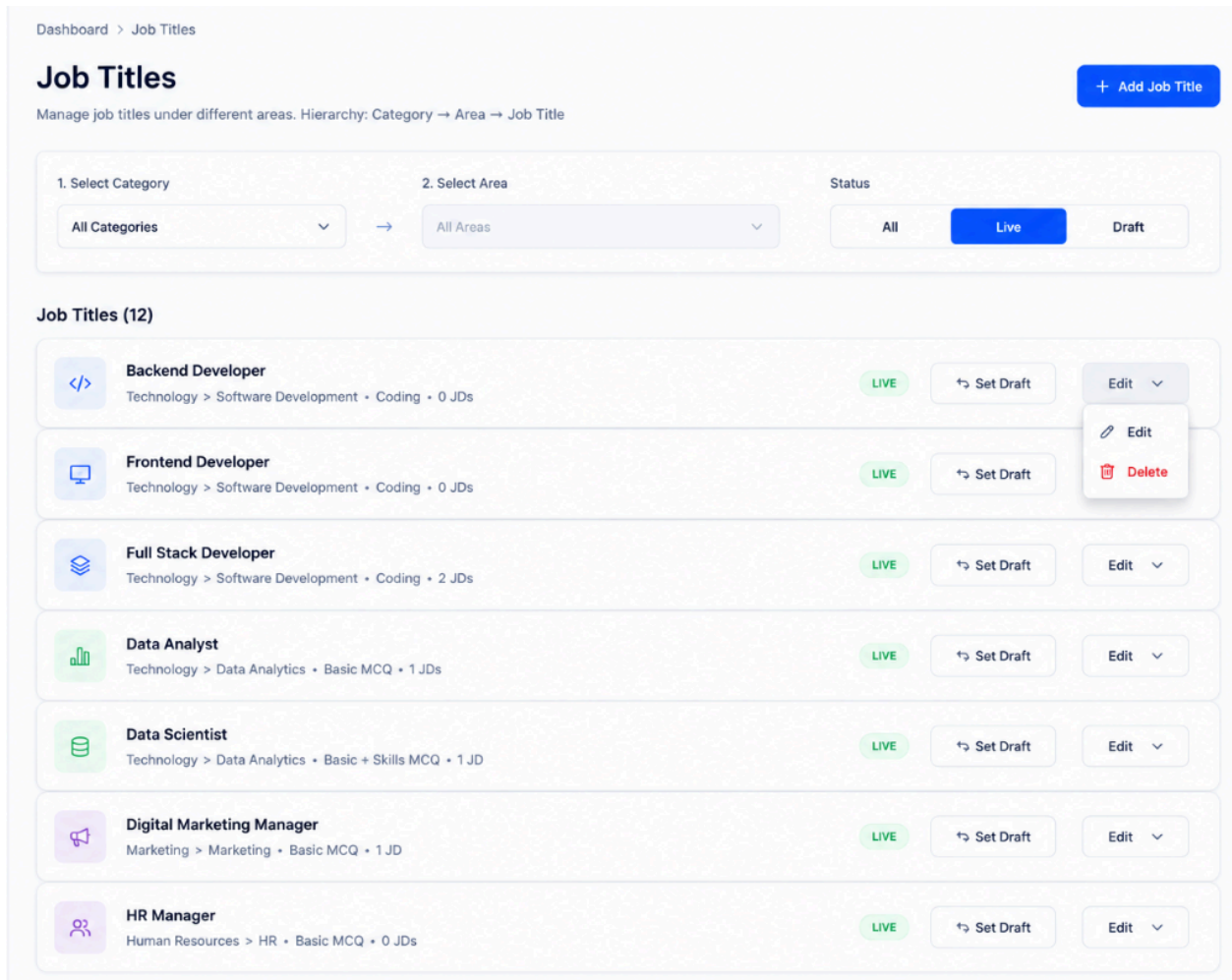
Area Name
Category
Top Titles/Careers
Status
Edit / Actions ▼

Important

Do not redesign the entire page.

The existing UI is already good, so treat these as **UX and functionality improvements to the current design**. Keep the current visual language and make these additions feel completely native to the existing interface.

JOB TITLES



Job Title Tab — Required Changes

The hierarchy must be:

Category → Area → Job Title

The UI and filtering should consistently follow this hierarchy.

1. Hierarchical Filters

Add the following filters:

- **Category dropdown**
- **Area dropdown**
- **Job Title listing**

The **Category** and **Area** filters must be placed on a separate line above the **Job Title** listing/content.

Filtering behavior

- The user must select a Category first.
- Before a Category is selected, the Area dropdown should be disabled or clearly indicate that a Category must be selected first.
- Once a Category is selected, the Area dropdown should populate with only Areas belonging to that Category.
- Include an “All” option where appropriate.
- Once an Area is selected, display only Job Titles belonging to that Area.
- If All Areas is selected for a Category, show all Job Titles belonging to Areas within that Category.

The flow should therefore be:

Select Category → Areas become available → Select Area → Job Titles are displayed

2. Status Filter

Keep the existing:

Live | Draft | All

- Live must always be selected by default whenever the page is opened/refreshed.
- Users can manually switch between Live, Draft, and All.
- The status filter must work together with the Category and Area filters.

For example:

Category: Technology + Area: Software Development + Status: Live
→ Show only Live Job Titles under Software Development → Technology.

Category: Technology + Area: All + Status: Live
→ Show all Live Job Titles across all Areas under Technology.

Category: All + Area: All + Status: Live
→ Show all Live Job Titles across all Categories.

3. Job Title Listing

Each Job Title row/card should clearly show the relevant hierarchy so the user can immediately understand where it belongs:

Job Title
Area → Category
Status
Edit / Actions ▼

Do not make the Category and Area feel like hidden metadata. They should be clearly visible while still keeping the UI clean.

4. Edit / Actions Dropdown

For all Category, Area, and Job Title management interfaces, use the existing Edit/action dropdown for management actions instead of adding multiple separate buttons to every row/card.

At minimum, the dropdown should contain:

- Edit
- Save as Draft / Move to Draft
- Go Live
- Delete

The actions should be status-aware.

For example:

Draft item

- Edit
- Go Live
- Delete

Live item

- Edit
- Save as Draft / Move to Draft
- Delete

All

- Show whichever actions are relevant to the item's current status.

Do not show contradictory actions such as Go Live for something that is already Live.

5. Delete

The Delete action must be available directly inside the existing Edit/Actions dropdown.

Do not add a separate Delete button to every card/list item.

When Delete is selected, use a confirmation dialog so the user does not accidentally remove a Category, Area, or Job Title.

For example:

Delete Job Title?

“This will remove this Job Title from the Area. This action cannot be undone.”

Then provide:

Cancel | Delete

Use the same confirmation pattern consistently across Category, Area, and Job Title management.

6. Add Job Title

If there is currently only a “+” icon for creating a Job Title, change it to a proper:

+ Add Job Title

The same principle should be used consistently for:

+ Add Category

+ Add Area

+ Add Job Title

7. Overall Hierarchy

The complete system should consistently follow:

Category

↓

Area

↓

Job Title

And the UI should enforce that relationship:

Select Category

↓

Areas belonging to that Category become available

↓

Select Area

↓

Job Titles belonging to that Area are displayed

Important UX Rule

Do not allow users to accidentally create or assign something to the wrong level of the hierarchy.

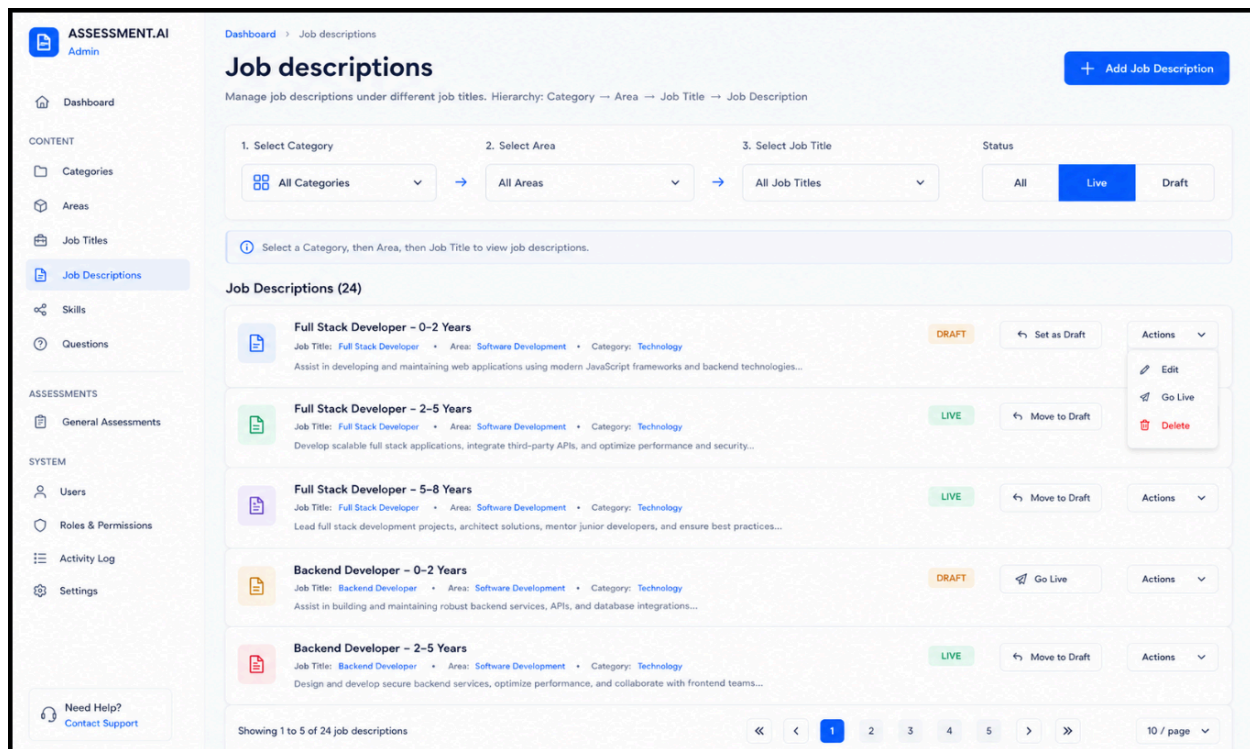
For example, when creating a Job Title:

Category → Area → Job Title

The Category should be selected first, then the Area dropdown should be filtered to that Category, and only then should the Job Title be created.

Keep the existing design and visual language unchanged. These should feel like natural extensions of the current UI, not a redesign.

JD



Job Description Tab — Required Changes

The **Job Description (JD)** tab must follow the same filtering hierarchy and behavior as the **Job Title** tab, with the JD coming directly under the selected Job Title.

The hierarchy should be:

Category → Area → Job Title → Job Description

Filtering

- Add a **Category** dropdown/filter.
- Add an **Area** dropdown/filter.
- Add a **Job Title** dropdown/filter.
- The filters should work **hierarchically**:
 - First select a **Category**.
 - Only the **Areas** belonging to that **Category** should then be available.
 - After selecting an Area, only the **Job Titles** belonging to that **Area** should be available.
 - After selecting a Job Title, show the **Job Descriptions** belonging to that **Job Title**.
- Include an **“All”** option where applicable.

- The **Category, Area, and Job Title filters** should be clearly separated from the **JD listing**, preferably on their own filter line/section.

Status Filter

Keep the existing:

Live | Draft | All

- **Live must always be selected by default** whenever the JD tab is opened.
- Users can manually switch to **Draft** or **All**.
- The status filter should work together with the Category → Area → Job Title hierarchy.

Add JD Button

- Replace the current “+” **icon-only button** with a proper “+ **Add Job Description**” button.

Edit / Delete

- Keep the existing **Edit** functionality.
- Add a **Delete** option inside the existing **Edit dropdown**.
- Do not create a separate Delete button on the JD cards.

Final Hierarchy

Category

↓

Area

↓

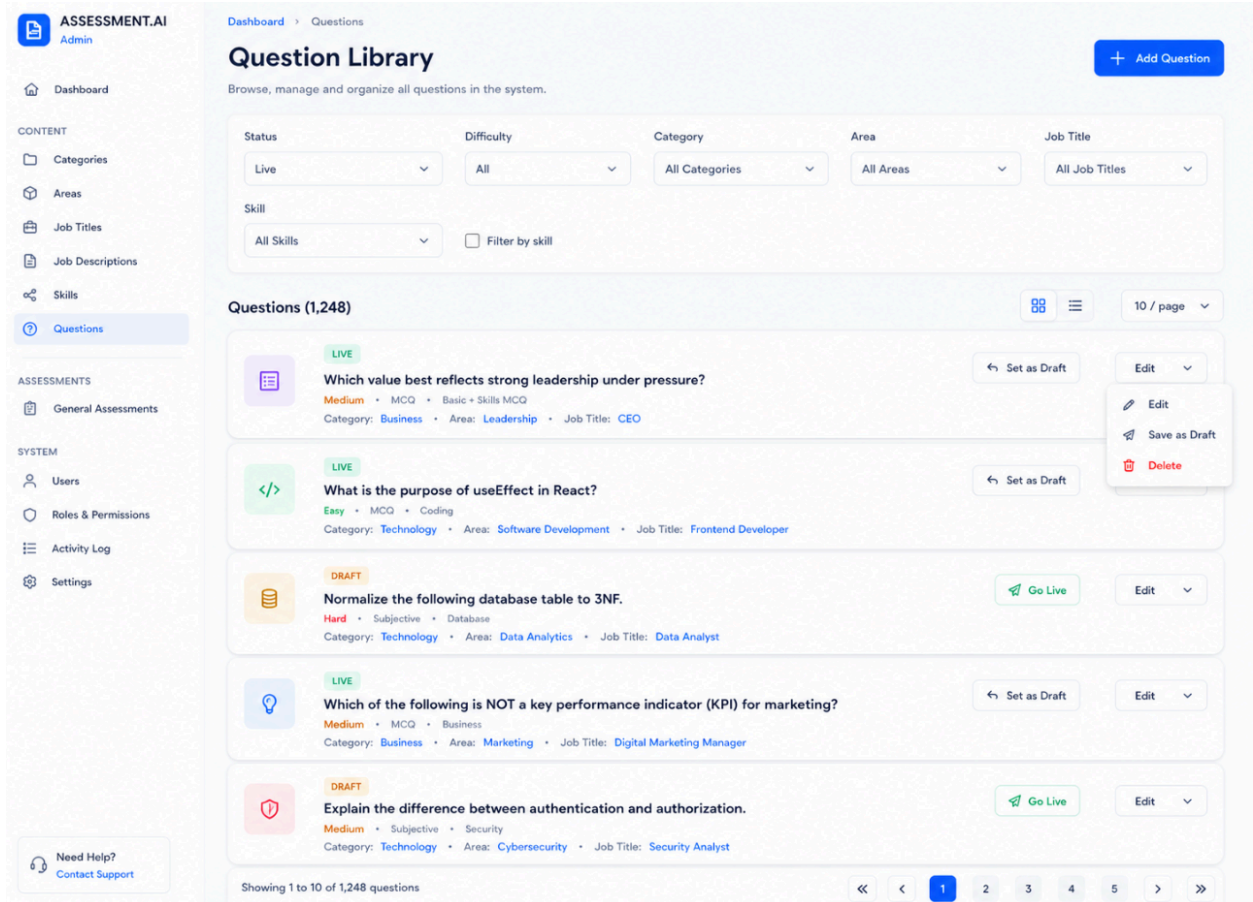
Job Title

↓

Job Description

The same filtering logic and behavior should be maintained consistently across **Categories, Areas, Job Titles, and Job Descriptions**.

question lib



Question Library — Required Changes

Please keep the **existing Question Library design exactly as it is** and make the following functional/UX additions.

1. Add Hierarchical Filters

In addition to the existing filters, add filters for:

- **Category**
- **Area**
- **Job Title**

These should follow the same hierarchy used throughout the system:

Category → Area → Job Title → Questions

Filtering should work like this:

1. Select a **Category**.
2. The **Area** dropdown should then show only Areas belonging to that Category.
3. Select an **Area**.

4. The **Job Title** dropdown should then show only Job Titles belonging to that Area.
5. Once a Job Title is selected, show only the **questions associated with that Job Title**.

Include **All** where appropriate.

The filters could be arranged cleanly in the existing filter row, for example:

Status ▼ | Difficulty ▼ | Category ▼ | Area ▼ | Job Title ▼ | Skill ▼

Make sure the filters don't make the existing UI feel overcrowded. If necessary, use a second filter row while maintaining the current design language.

2. Filters Should Work Together

All filters should work together rather than independently.

For example:

Category: Technology

→ **Area: Software Development**

→ **Job Title: Full Stack Developer**

→ Show only questions associated with Full Stack Developer.

And:

Category: All → Area: All → Job Title: All

→ Show questions across the entire question library.

The existing **Status** and **Difficulty** filters should continue working alongside these hierarchy filters.

3. Edit Question

Every question should have an **Edit** action.

Keep the current clean card/list design, but provide an **Edit / Actions dropdown** for each question rather than adding large buttons that clutter the cards.

The dropdown should contain at minimum:

- **Edit**
- **Delete**

If the question has draft/live functionality, also include the relevant status action:

- **Save as Draft / Move to Draft**
- **Go Live**

Actions should be status-aware.

For example:

Live question

- Edit
- Save as Draft
- Delete

Draft question

- Edit
- Go Live
- Delete

4. Delete Question

Add **Delete** directly inside the question's Edit/Actions dropdown.

Do **not** add a separate Delete button to every question card.

When Delete is selected, show a confirmation modal:

Delete Question?

“Are you sure you want to delete this question? This action cannot be undone.”

Cancel | Delete

5. Question Card Information

Keep the current card design, but make sure the question's hierarchy is clear.

Ideally the metadata should communicate something like:

Which value best reflects strong leadership under pressure?

Medium · MCQ · Basic + Skills MCQ

Category: Business

Area: Leadership

Job Title: CEO

This makes it immediately clear where the question belongs without requiring the user to open it.

Final hierarchy

The Question Library should follow the same structure as the rest of the system:

Category



Area



Job Title



Question

And the user should be able to:

Filter → Find Question → Edit → Save as Draft / Go Live → Delete

Keep the existing visual design, spacing, typography, cards, and overall UI intact. These are additions to the existing Question Library, **not a redesign**.

also delete questions

Assessment AI — Backend, Database &
User Flow Imp...

Assessment AI — Backend, Database & User Flow Implementation

You are working on an existing **Assessment AI** application.

Your task is to connect the complete assessment user flow to the backend and database so that every assessment type works end-to-end reliably.

Do not treat individual screens as isolated features.

The system has **exactly four assessment flows**, and this distinction is critical:

1. General Assessment
2. Basic MCQ
3. Basic + Skills MCQ
4. Coding Assessment

The backend, database schema, APIs, assessment-generation logic, question selection, replacement logic, submissions, and results must all support these four flows correctly.

Do not merge the four flows into one generic flow if doing so changes their intended behavior.

1. First Understand the Existing Codebase

Before making changes:

- Inspect the existing frontend.
- Inspect all existing API routes.
- Inspect the Prisma/database schema.
- Inspect existing assessment-related models.
- Inspect the Python/FastAPI service.
- Inspect existing OpenAI/AI generation functions.
- Inspect authentication and user identification.
- Inspect any existing question library implementation.
- Inspect JD-related implementation.
- Inspect skill-related implementation.
- Inspect coding assessment infrastructure, if already present.

Reuse existing working functionality where possible.

Do not create duplicate models, endpoints, services, or utilities when equivalent functionality already exists.

Do not break existing functionality.

2. Architecture

The application architecture is:

- Frontend / Main Backend: Next.js + TypeScript
- API layer: Next.js API Routes
- Database ORM: Prisma
- Database: PostgreSQL
- AI / processing service: Python + FastAPI
- LLM provider: OpenAI

Use the responsibilities appropriately.

Next.js backend should handle:

- authentication
- authorization
- user ownership
- assessment lifecycle
- database operations
- question-library queries
- question-history checks
- JD library operations
- assessment configuration
- saving assessment attempts
- saving answers
- calculating or coordinating results
- communication with FastAPI

FastAPI should handle:

- AI question generation
- AI coding-question generation
- JD generation where applicable
- JD skill extraction
- structured AI processing

- deterministic processing/scoring where appropriate

PostgreSQL should persist durable application data.

Do not use AI memory as a replacement for database persistence.

3. Core Assessment Hierarchy

The data hierarchy should support:

Category

- Area of Interest
- Job Title
- Assessment Type / Flow
- Assessment Configuration
- JD
- Skills Being Tested
- Questions
- Assessment Attempt
- User Answers
- Results

Example:

Commerce

- Marketing
- Social Media Manager
- Basic + Skills MCQ

Another example:

Technology

- Software Development
- Frontend Developer
- Coding Assessment

General assessments do not require Job Titles.

Example:

General / Aptitude

- General Assessment

4. Categories and Areas of Interest

The first screen shows Categories.

Examples:

Commerce

- Marketing
- Finance
- HR

Technology

- Software Development
- Data Analytics
- Cyber Security

Healthcare

- relevant healthcare areas

Categories and Areas of Interest must come from the database.

Do not hardcode these relationships in the frontend.

Each Area of Interest must have enough metadata for the backend to determine whether it is:

- General Assessment
- Role-Based Assessment

When the user selects an Area of Interest, the backend determines what happens next.

5. Flow Selection

Option A — General Assessment

Examples:

- Aptitude
- Logical Reasoning
- Quantitative Ability
- Pattern Recognition

These are not associated with a Job Title.

Therefore:

Area of Interest

→ Assessment Setup

The user must NOT see Job Title selection.

Option B — Role-Based Assessment

Examples:

- Marketing
- Finance
- HR
- Software Development

The user proceeds:

Area of Interest

→ Job Title Selection

Job Titles must be retrieved from the database based on the selected Area of Interest.

Example:

Marketing:

- Social Media Manager
- Content Writer
- SEO Specialist

Each Job Title must have an assigned assessment flow/type.

Possible values:

- BASIC_MCQ
- BASIC_SKILLS_MCQ

- CODING

The frontend should not decide the flow using hardcoded Job Title names.

The backend/database configuration should determine it.

6. FLOW 1 — GENERAL ASSESSMENT

This is the first of the four assessment flows.

Setup

After selecting a General Area of Interest, show:

- Difficulty
- Number of Questions
- Preview On/Off

Do NOT show:

- Job Title
- Experience Level
- JD
- Skills To Focus On

User clicks:

Generate Assessment

Question Selection

Search the Question Library based on:

- Area of Interest
- Difficulty
- Question Type
- User's question history

Questions answered correctly by the same user during the previous **30 days** should not normally be selected.

Priority:

1. Questions never attempted by this user
2. Questions previously answered incorrectly
3. AI-generated questions

If enough eligible library questions exist:

Use library questions.

Example:

Requested = 20

Eligible library questions = 20

Use all 20.

If insufficient library questions exist:

Example:

Requested = 20

Eligible library questions = 14

Use:

- 14 library questions
- 6 AI-generated questions

AI-generated questions do not need long-term question-history tracking.

Preview

If Preview = ON:

Display all generated questions.

Allow:

Replace Question

Do NOT display:

- correct answer
- explanation
- solution

Then:

Start Test

If Preview = OFF:

Start the assessment immediately.

Completion

After submission show:

- Final Score
- Final Percentage
- Skill-based score if applicable

Persist:

- assessment attempt
- answers
- whether each library question was correct or incorrect

This history must later be used by the 30-day filtering logic.

7. FLOW 2 — BASIC MCQ

This is the second assessment flow.

The user selected a role-based Job Title configured as:

BASIC_MCQ

Setup

User selects:

- Difficulty
- Experience Level
- Number of Questions
- Preview On/Off

User can also choose:

Create Your Own Assessment

If Create Your Own Assessment is selected, redirect into the configuration described in Flow 3.

User clicks:

Continue

8. JD SCREEN — FLOW 2

The assessment requires a Job Description.

If a default JD exists for the selected Job Title, automatically show it.

The user has three options:

Choose Existing JD

Select a JD from the JD Library.

Generate JD

Generate a Job Description with AI based on:

- Job Title
- Area of Interest
- Experience Level
- relevant metadata

Paste JD

Allow the user to paste their own Job Description.

The user must finalize one JD before proceeding.

9. JD STORAGE

Support different JD sources.

Possible source enum:

- LIBRARY
- AI_GENERATED
- USER_PASTED

Store the source for traceability.

However:

The JD source must NEVER affect question-library eligibility.

Whether the JD is:

- from the library
- AI generated
- pasted

the extracted skills must still be usable to search the Question Library.

10. JD SKILL EXTRACTION

After the JD is finalized, extract skills.

Example:

Job Title:

SEO Specialist

JD may produce:

- SEO
- Content Writing
- Social Media Marketing

- Analytics

Display:

Skills Being Tested

- SEO
- Content Writing
- Social Media Marketing
- Analytics

For Flow 2 these skills are review-only.

The user cannot manually edit them.

11. SKILL NORMALIZATION

Skills must reference a central Skills table/library.

Do not store arbitrary variations such as:

- React JS
- ReactJS
- React.js
- react

as separate skills.

Use canonical skills.

Example:

React

Skills should have IDs and normalized names.

AI-extracted skills should be mapped to existing skills whenever possible.

This is important because question-library queries depend on skill IDs/tags.

12. QUESTION GENERATION — FLOW 2

Search Question Library using:

- Skill Tags
- Difficulty
- Experience Level where applicable
- Job Title tags where appropriate

Apply 30-day question-history filtering.

Priority should favor questions matching the assessment skills.

Example:

Requested = 20

Eligible library questions = 15

Result:

- 15 library questions
- 5 AI-generated questions

Every AI-generated question MUST include associated skill IDs/tags.

Example:

Question:

"What is the difference between on-page and off-page SEO?"

Skill:

SEO

This association is mandatory because the final result needs skill-based scoring.

13. FLOW 3 — BASIC + SKILLS MCQ

This is the third assessment flow.

It follows the same core process as Basic MCQ with one important additional capability:

Skills To Focus On

Setup

Show:

- Difficulty
- Experience Level
- Number of Questions
- Skills To Focus On
- Preview On/Off

Multiple Skills To Focus On can be selected.

Examples:

- SEO
 - Recruitment
 - Google Ads
 - Excel
-

14. SKILLS TO FOCUS ON RULE

Users cannot create arbitrary skills.

The Skills To Focus On field must search/select from the Skills database.

If the skill already exists:

Allow selection.

If the skill does not exist:

Do NOT allow the user to add a random custom skill.

The UI can display something such as:

"No matching skill available."

Do not silently create new skills from arbitrary user text.

15. JD — FLOW 3

Use the same JD flow as Basic MCQ:

- Choose Existing JD
- Generate JD
- Paste JD

Extract JD skills.

16. FINAL SKILLS — FLOW 3

Combine:

User-selected Skills

Example:

- Google Ads
- Content Writing

with:

JD Skills

Example:

- SEO
- Analytics
- Social Media Marketing

Final Skills Being Tested:

- SEO
- Analytics
- Social Media Marketing
- Google Ads
- Content Writing

Remove duplicates.

Every final skill should reference an actual Skill record.

Store the final assessment-skill mapping so the final assessment can later be reconstructed exactly.

17. QUESTION PRIORITY — FLOW 3

Question-generation priority:

1. User Selected Skills
2. JD Skills
3. Job Title Tags

Try to distribute questions across the final skills rather than accidentally generating all questions from one skill.

Still apply:

- Difficulty
- Experience Level
- 30-day user question history

If library questions are insufficient:

Generate the remaining questions using AI.

Every AI-generated question MUST have skill tags attached.

18. CREATE YOUR OWN ASSESSMENT

From applicable Role-Based assessment screens, the user may select:

Create Your Own Assessment

This should enter the Flow 3 configuration experience.

It should not create an unrelated fifth assessment flow.

There are still exactly four assessment flows.

The custom option is a way of configuring the **Basic + Skills MCQ** flow.

19. FLOW 4 — CODING ASSESSMENT

This is the fourth assessment flow.

For Job Titles configured to support coding assessments, show:

- Difficulty
- Experience Level
- Number of Questions
- Skills To Focus On
- Assessment Mode
- Preview On/Off

Assessment Mode:

- MCQ
- Coding

User clicks:

Continue

20. JD — FLOW 4

Use the same JD system.

Options:

- Choose Existing JD
- Generate JD
- Paste JD

Extract skills.

Combine:

- User Selected Skills
- JD Skills

and use Job Title skill tags where appropriate.

Example:

- React
- JavaScript
- Node.js
- SQL
- MongoDB

Display the final:

Skills Being Tested

before assessment generation.

21. FLOW 4 — MCQ MODE

If:

Assessment Mode = MCQ

Use the same question-selection rules as Flow 3.

Search based on:

1. Selected Skills
2. JD Skills
3. Job Title Tags
4. Difficulty
5. Experience Level where applicable

Then:

Library Questions

→ AI fallback if necessary.

22. FLOW 4 — CODING MODE

If:

Assessment Mode = Coding

Search the Coding Question Library using:

- Selected Skills
- JD Skills
- Job Title Tags
- Difficulty
- Experience Level where applicable

Apply user-history logic where applicable.

If enough eligible coding challenges exist:

Use library challenges.

Otherwise:

Generate the remaining coding challenges with AI.

Example:

Requested = 5

Eligible library coding challenges = 3

Generate:

- 3 library coding challenges
- 2 AI coding challenges

AI-generated coding challenges must contain enough structured information to be executed and evaluated safely.

23. CODING QUESTION STRUCTURE

A coding challenge should support fields similar to:

- id
- title
- problemStatement
- difficulty
- skillId / skill tags
- starterCode
- language
- constraints
- test cases
- expected outputs
- source
- createdAt

Library coding questions must have predefined test cases.

AI-generated coding questions must also generate structured test cases before being given to the user.

Never expose hidden expected solutions or private test cases on the frontend.

24. CODING ASSESSMENT SCREEN

Display:

- Challenge Title
- Problem Statement
- Code Editor
- Language where applicable
- Run Code
- Submit Code

Run Code

Should execute against safe/public test cases where applicable.

Submit Code

Should evaluate against the complete test-case set.

Persist:

- submitted code

- result
- passed tests
- failed tests
- final correctness

The frontend must not be responsible for deciding whether the answer is correct.

Evaluation must happen server-side.

25. PREVIEW — CODING

If Preview = ON display:

- Coding Challenge Title
- Problem Statement

Allow:

Replace Question

Do NOT show:

- solution
- answer
- expected implementation
- hidden test cases

If Preview = OFF:

Start immediately.

26. QUESTION REPLACEMENT — ALL FOUR FLOWS

Question replacement must be implemented consistently.

When Replace Question is clicked, exclude the current question and preferably all questions already present in the same assessment.

Priority:

1. Question never attempted by the user
2. Question previously answered incorrectly
3. AI-generated replacement

Avoid questions answered correctly in the previous 30 days.

Never return the exact same question that the user just replaced.

Do not accidentally create duplicate questions inside the same assessment.

Replacement must maintain:

- assessment type
- question type
- difficulty
- appropriate skill
- appropriate role/category context

For skill-focused assessments, try to replace a question with another question testing the same skill.

27. QUESTION HISTORY — 30 DAY RULE

This rule is important.

For Question Library questions, track whether a user answered:

- correctly
- incorrectly

If the user answered a library question correctly during the previous 30 days:

Exclude it from normal generation and replacement where possible.

After 30 days it becomes eligible again.

Questions previously answered incorrectly may be reused.

Never-attempted questions have highest priority.

AI-generated questions do not require long-term question-history tracking.

The history filtering must happen server-side.

Do not download the full question library to the frontend and filter there.

28. ASSESSMENT SNAPSHOT

Once an assessment is generated, store a snapshot of what was actually selected.

An assessment should retain information such as:

- userId
- assessment flow
- assessment mode
- categoryId
- areaOfInterestId
- jobId if applicable
- JD/reference
- difficulty
- experienceLevel if applicable
- requestedQuestionCount
- previewEnabled
- selectedSkillIds
- jdSkillIds
- finalSkillIds
- generated questions
- question order
- assessment status
- createdAt
- startedAt
- completedAt

Do not dynamically rebuild an already-started assessment from the current Question Library.

Once generated, the assessment should remain stable.

29. ASSESSMENT STATUS

Use a clear lifecycle.

For example:

- CONFIGURING
- GENERATED
- PREVIEW
- IN_PROGRESS
- COMPLETED
- ABANDONED

Only allow valid transitions.

Example:

CONFIGURING
→ GENERATED
→ PREVIEW
→ IN_PROGRESS
→ COMPLETED

or:

CONFIGURING
→ GENERATED
→ IN_PROGRESS
→ COMPLETED

when preview is disabled.

Prevent invalid operations such as:

- submitting an assessment before starting
- modifying configuration after completion
- replacing questions after test submission
- submitting answers to another user's assessment

30. DATABASE DESIGN

Review the existing Prisma schema first.

Adapt existing tables if suitable.

Do not blindly create duplicates.

The data model must conceptually support entities similar to:

User

Existing application user.

Category

Fields such as:

- id
- name
- slug
- active

AreaOfInterest

- id
- categoryId
- name
- slug
- assessmentClassification
- active

Possible classification:

- GENERAL
- ROLE_BASED

JobTitle

- id
- areaOfInterestId
- name
- slug
- assessmentFlow
- active

Possible assessmentFlow:

- BASIC_MCQ
- BASIC_SKILLS_MCQ
- CODING

Skill

- id
- name
- normalizedName
- slug
- active

JobTitleSkill

Many-to-many mapping.

JD

Possible fields:

- id
- jobTitleId
- title
- content
- source
- createdByUserId where applicable
- active
- createdAt

JDSkill

JD ↔ Skill mapping.

Question

For MCQ library questions.

Possible fields:

- id
- questionText
- difficulty
- source
- active
- createdAt

Question source examples:

- LIBRARY
- AI

For long-term storage, only permanent library questions need to exist globally.

All questions can instead be stored as assessment-specific snapshots.

QuestionSkill

Question ↔ Skill mapping.

QuestionArea

Optional relation to Area of Interest.

QuestionJobTitle

Optional relation to Job Title.

QuestionOption

- id
- questionId
- text
- isCorrect

Correct-answer information must never be sent to the assessment-taking frontend before submission.

CodingQuestion

- id
- title
- problemStatement
- difficulty
- starterCode
- language
- source
- active

CodingQuestionSkill

CodingQuestion ↔ Skill.

CodingTestCase

- id
- codingQuestionId

- input
- expectedOutput
- visibility

Possible visibility:

- PUBLIC
- HIDDEN

Assessment

Represents a generated assessment instance.

AssessmentSkill

Stores the final Skills Being Tested.

Include a source field where useful:

- USER_SELECTED
- JD
- JOB_TITLE

A skill could have multiple origins, so design this carefully.

AssessmentQuestion

Snapshot/order of questions assigned to one assessment.

Possible fields:

- assessmentId
- libraryQuestionId nullable
- codingQuestionId nullable
- sequence
- source
- skillId
- questionSnapshot JSON where appropriate
- replacement metadata

This is important for AI-generated questions because they do not necessarily belong in the permanent Question Library.

AssessmentAttempt

If Assessment itself represents configuration separately from a user's attempt, model attempts appropriately.

Otherwise make the ownership/lifecycle explicit.

UserAnswer

Possible fields:

- assessmentQuestionId
- userId
- selectedOptionId
- submittedCode where applicable
- isCorrect
- score
- submittedAt

UserQuestionHistory

This can either be an explicit table or derived efficiently from UserAnswer records.

It must support fast queries for:

- user
- library question
- last correct answer date

The exact implementation can differ as long as the 30-day rule is reliable.

31. DATABASE RELATIONSHIP PRINCIPLE

Do not store important relationships only as strings.

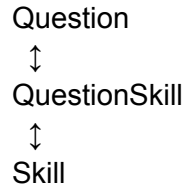
Prefer IDs and foreign keys.

For example, do not rely on:

`question.skill = "React"`

when a canonical Skill entity exists.

Use relationships such as:



This prevents inconsistent skill matching.

32. QUESTION LIBRARY QUERY LOGIC

Create centralized server-side question-selection logic.

Do not repeat slightly different query logic in every API route.

The selection engine should take parameters similar to:

- userId
- assessmentFlow
- assessmentMode
- categoryId
- areaOfInterestId
- jobIdId
- selectedSkillIds
- jdSkillIds
- finalSkillIds
- difficulty
- experienceLevel
- requestedCount
- excludedQuestionIds

It should return the best eligible library questions.

Then calculate:

```
remainingCount = requestedCount - libraryQuestions.length
```

If:

```
remainingCount > 0
```

call AI generation for exactly that remaining amount.

33. AI QUESTION GENERATION

AI output must be structured.

Never depend on unvalidated free-form LLM text.

For MCQ generation require structured output similar to:

```
{
  "questions": [
    {
      "question": "...",
      "difficulty": "MEDIUM",
      "skillId": "...",
      "options": [
        {
          "text": "...",
          "isCorrect": false
        }
      ]
    }
  ]
}
```

Validate AI output server-side.

Check:

- requested number of questions
- valid skill
- valid difficulty
- exactly one correct option where appropriate
- sufficient answer options
- no empty values
- no obvious duplicates

Never trust raw AI output directly.

34. AI CODING QUESTION GENERATION

Coding output must also use a strict validated structure.

Example:

```
{
  "questions": [
    {
      "title": "...",
      "problemStatement": "...",
      "skillId": "...",
      "difficulty": "MEDIUM",
      "language": "javascript",
      "starterCode": "...",
      "testCases": [
        {
          "input": "...",
          "expectedOutput": "...",
          "visibility": "HIDDEN"
        }
      ]
    }
  ]
}
```

Validate before saving/using.

35. QUESTION COUNT GUARANTEE

The system must never silently return fewer questions than requested.

Example:

User requests 20.

Library returns 13.

AI generation must attempt to generate 7.

Final assessment should contain 20 unique questions.

If question generation genuinely fails, return a proper backend error rather than starting an incomplete assessment without explanation.

36. DUPLICATE PREVENTION

Within one assessment:

- no duplicate library question IDs
- no repeated AI questions
- no duplicate replacement question
- no repeated coding challenge

When generating AI questions, compare normalized question text against already-selected questions where practical.

37. RESULTS — VERSION 1

After assessment completion show only:

- Final Score
- Final Percentage
- Skill-Based Score

No additional analytics are required for V1.

Do NOT add:

- AI feedback
- detailed performance reports
- personality analysis
- candidate recommendations
- advanced analytics

unless already required elsewhere in the product.

38. SKILL-BASED SCORE

Every scored question must be associated with at least one skill where skill scoring is expected.

Calculate:

$$\begin{aligned} \text{Skill Percentage} = & \\ \text{Correct Questions For Skill} & \\ / & \\ \text{Total Questions For Skill} & \\ \times 100 & \end{aligned}$$

Example:

SEO:

3 correct / 4 questions

= 75%

JavaScript:

4 correct / 5 questions

= 80%

Do not calculate skill percentages on the frontend from untrusted values.

Backend should return the final result.

39. GENERAL ASSESSMENT SKILLS

If general assessments need skill-based scoring, treat areas such as:

- Logical Reasoning
- Quantitative Ability
- Pattern Recognition

as measurable assessment skills/domains.

If the existing product does not require skill-based scores for General Assessments, keep the implementation compatible without forcing artificial skills.

40. SECURITY

Never send these fields to the frontend before an answer is submitted:

- correct answer
- `isCorrect`
- expected answer
- hidden test case
- coding solution
- scoring secrets

Users must only be able to access:

- their own generated assessments
- their own attempts
- their own results
- permitted shared library metadata

Validate assessment ownership on every relevant backend endpoint.

Do not trust:

- `userId` from frontend
- `score` from frontend
- `isCorrect` from frontend

Derive user identity from authenticated session/token.

Calculate correctness server-side.

41. API DESIGN

Inspect existing routes first and reuse conventions.

The backend should conceptually support operations such as:

Taxonomy

- Get Categories
- Get Areas by Category
- Get Job Titles by Area
- Get Skills
- Search Skills

Assessment Configuration

- Create Assessment
- Update Assessment Configuration
- Resolve Assessment Flow

JD

- Get JD Library
- Select JD
- Generate JD
- Save/Use Pasted JD
- Extract JD Skills

Assessment Generation

- Generate Assessment
- Replace Question
- Start Assessment

Submission

- Submit MCQ Answer / answers
- Run Code
- Submit Code
- Complete Assessment

Results

- Get Assessment Result

Do not necessarily create one endpoint for every bullet if the existing architecture has a cleaner structure.

The important requirement is correct separation of responsibilities and predictable API behavior.

42. TRANSACTIONS

Use database transactions for operations where partial writes could corrupt an assessment.

Examples:

Assessment generation may involve:

1. creating Assessment
2. attaching skills
3. attaching library questions
4. storing AI-generated question snapshots
5. updating assessment status

These operations should not leave a half-created assessment if something fails.

Use Prisma transactions where appropriate.

43. IDEMPOTENCY / DOUBLE CLICK PROTECTION

Prevent duplicate assessments if the user double-clicks:

Generate Assessment

Prevent duplicate submission if:

Submit Test

is triggered twice.

Frontend loading states help, but backend protection is still required.

Do not rely exclusively on UI button disabling.

44. CONCURRENCY

Be careful if two requests attempt to:

- replace the same question
- submit the same assessment
- generate the same assessment
- finish the same attempt

Validate current assessment state before modification.

45. VALIDATION

Every API must validate input.

Examples:

Difficulty must be one of:

- EASY
- MEDIUM
- HARD

Number of Questions must be within configured system limits.

Experience level must be valid.

Selected Skill IDs must exist.

Job Title must belong to selected Area of Interest.

Area must belong to selected Category.

Selected JD must be valid for its expected context.

Assessment Flow must match the Job Title configuration.

Coding mode should only be available where supported.

Never trust frontend relationships.

46. FRONTEND STATE

The frontend should receive enough backend information to determine the next valid step.

Do not duplicate business rules between frontend and backend.

For example:

After Area selection the API can return:

```
{
  "areald": "...",
  "classification": "ROLE_BASED",
  "nextStep": "JOB_TITLE"
}
```

After Job Title selection:

```
{
  "jobTitleId": "...",
  "assessmentFlow": "BASIC_SKILLS_MCQ",
  "nextStep": "ASSESSMENT_SETUP"
}
```

The backend/database is the source of truth.

47. FOUR-FLOW STATE MACHINE

The final implementation must preserve these paths.

FLOW 1

Categories

- Area of Interest
- GENERAL detected
- General Setup
- Generate Assessment
- Library Questions
- AI fallback if required
- Preview OR Start
- Assessment
- Submit

→ Results

FLOW 2

Categories

- Area of Interest
- ROLE_BASED
- Job Title
- BASIC_MCQ
- Setup
- JD
- Skills Extraction
- Skills Being Tested
- Question Library
- AI fallback
- Preview OR Start
- Assessment
- Submit
- Results

FLOW 3

Categories

- Area of Interest
- ROLE_BASED
- Job Title
- BASIC_SKILLS_MCQ
- Setup + Skills To Focus On
- JD
- JD Skill Extraction
- Merge Selected Skills + JD Skills
- Final Skills Being Tested
- Question Library
- AI fallback
- Preview OR Start
- Assessment
- Submit
- Results

FLOW 4

Categories

- Area of Interest
- ROLE_BASED
- Job Title
- CODING
- Setup + Skills + Mode

IF MODE = MCQ:

- JD
- Final Skills
- MCQ Library
- AI fallback
- Preview OR Start
- Assessment
- Results

IF MODE = CODING:

- JD
- Final Skills
- Coding Library
- AI fallback
- Preview OR Start
- Coding Assessment
- Run / Submit Code
- Results

These four paths are mandatory.

48. EDGE CASES

Handle at minimum:

- Category has no Areas
- Area has no Job Titles
- Job Title has no default JD
- JD has no extractable skills
- AI extracts skill not present in Skill Library
- User selects duplicate skills
- Library has zero eligible questions
- User correctly answered all available library questions in previous 30 days

- Library only partially satisfies requested question count
- AI question generation fails
- AI returns malformed JSON
- AI generates duplicate questions
- assessment generation request is retried
- question replacement is clicked multiple times
- replacement pool is exhausted
- user refreshes during preview
- user refreshes during assessment
- user leaves and returns
- assessment is already completed
- user attempts to submit twice
- user tries to access another user's assessment
- JD gets modified after an assessment was already generated
- Question Library question gets edited after an assessment starts
- Skill name changes after an assessment starts

Use snapshots where necessary so old assessments remain historically correct.

49. DATABASE SEEDING / ADMIN DATA

Categories, Areas, Job Titles, Skills, default JDs, and library questions should be maintainable as database data.

Do not hardcode production taxonomy in React components.

If seed files already exist, extend them cleanly.

Ensure relations are created using stable IDs/slugs.

50. PERFORMANCE

Question selection should be performed efficiently in PostgreSQL/Prisma.

Do not:

1. retrieve the entire Question Library
2. retrieve the entire user history

3. filter everything in JavaScript

Use database filtering wherever practical.

Add indexes for commonly queried fields/relationships such as:

- categoryId
- areaOfInterestId
- jobId
- skillId
- difficulty
- userId
- questionId
- completedAt / answeredAt
- assessmentId

Choose exact indexes based on the final Prisma schema.

51. SOURCE TRACKING

For every assessment question, know whether it came from:

- QUESTION_LIBRARY
- AI_GENERATED
- CODING_LIBRARY
- AI_CODING_GENERATED

This is required so that:

- 30-day tracking applies correctly
 - AI questions are not incorrectly added to permanent history
 - replacement logic behaves correctly
-

52. AI QUESTIONS ARE TEMPORARY

AI-generated questions should NOT automatically become permanent Question Library questions.

They belong to the generated assessment/attempt and may be stored as an assessment snapshot for persistence.

They do not need the same cross-assessment 30-day history tracking as library questions.

Do not accidentally pollute the permanent Question Library with every AI generation.

53. IMPLEMENTATION APPROACH

Work in this order:

Phase 1 — Audit

Understand:

- current schema
- current frontend
- current APIs
- current FastAPI implementation
- existing reusable functionality

Phase 2 — Data Model

Fix or extend:

- Categories
- Areas
- Job Titles
- Skills
- JDs
- Questions
- Coding Questions
- Assessments
- Answers
- Question History

Phase 3 — Backend

Implement:

- taxonomy APIs
- flow resolution

- JD handling
- skill extraction
- centralized question selection
- AI fallback
- replacement logic
- assessment lifecycle
- submission
- scoring

Phase 4 — Frontend Integration

Connect every screen to actual backend responses.

Remove hardcoded fake data when database-backed data is expected.

Phase 5 — Testing

Test all four flows independently.

54. REQUIRED TEST SCENARIOS

Do not consider this feature complete without testing these scenarios.

Flow 1

General Assessment with:

- enough library questions
- insufficient library questions
- preview ON
- preview OFF
- replacement
- 30-day history filtering

Flow 2

Basic MCQ with:

- Existing JD
- AI JD
- Pasted JD

- library-only questions
- mixed library + AI
- preview ON/OFF
- replacement

Flow 3

Basic + Skills with:

- one selected skill
- multiple selected skills
- overlapping JD + selected skill
- nonexistent skill search
- partial library coverage
- AI fallback
- correct skill scoring

Flow 4

Test both:

- MCQ mode
- Coding mode

Coding mode must test:

- coding library question
- AI-generated coding question
- Run Code
- Submit Code
- public tests
- hidden tests
- preview
- replacement
- final score

55. DATABASE CONSISTENCY TEST

Verify:

- no orphan records
- correct foreign keys

- correct cascade/restrict behavior
 - no duplicate AssessmentQuestion entries
 - no duplicate AssessmentSkill mappings
 - completed assessment cannot be mutated improperly
 - AI question snapshot survives refresh/reload
 - assessment can be reconstructed completely from database state
-

56. DO NOT DO THESE

Do not:

- hardcode Job Title → assessment flow mappings in frontend
 - hardcode skill relationships
 - trust frontend score calculations
 - send correct answers before assessment completion
 - put all business logic inside React components
 - create random skills from arbitrary user input
 - require AI for questions when library questions exist
 - permanently save every AI-generated question into the Question Library
 - ask correctly answered library questions again within 30 days unless unavoidable
 - return duplicate questions in one assessment
 - change assessment questions after the test starts unless explicitly supported
 - rebuild an existing assessment every time the page refreshes
 - lose AI-generated questions after refresh
 - let JD source determine whether library questions can be used
 - create a fifth assessment flow for Create Your Own Assessment
 - silently return fewer questions than requested
 - mix coding questions and MCQs accidentally
 - expose hidden coding test cases
 - calculate authoritative scores on the client
-

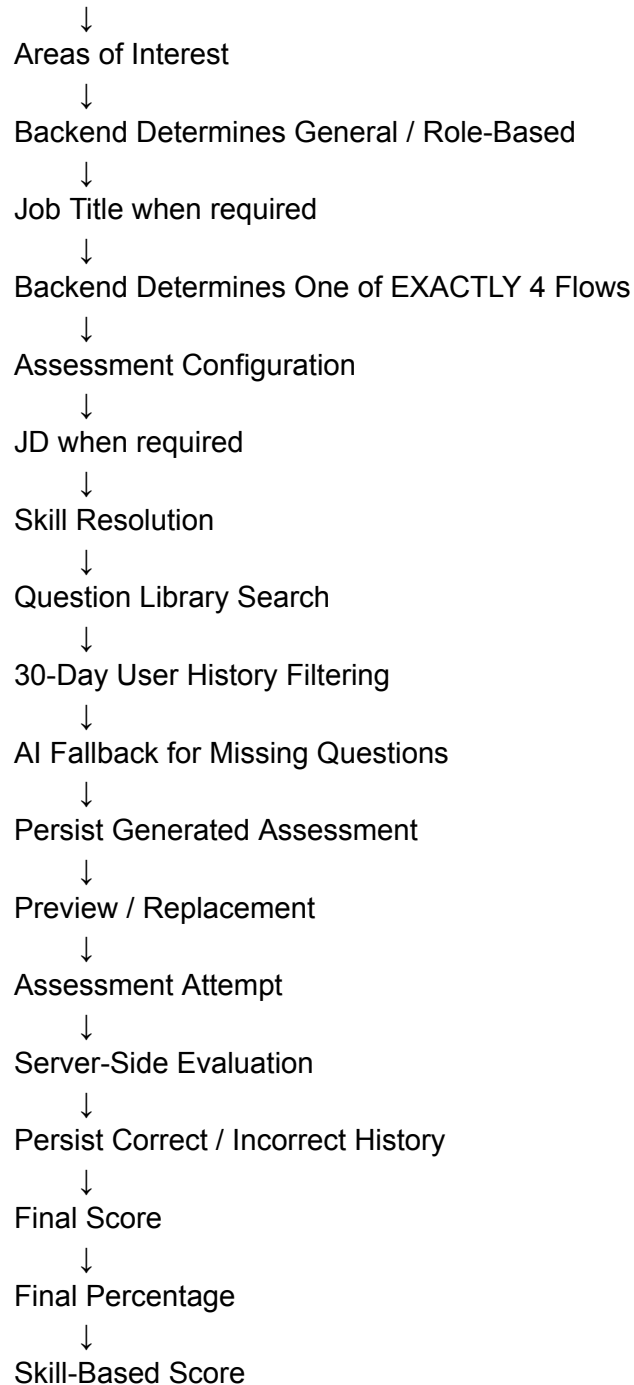
57. EXPECTED FINAL RESULT

The final product must behave as one connected system:

DATABASE CONFIGURATION



Categories



The database must be the source of truth for taxonomy and relationships.

The backend must be the source of truth for business logic, flow selection, assessment state, question selection, correctness, and scoring.

The frontend should primarily render the current state and send user actions.

58. BEFORE WRITING CODE

Before implementing changes, provide:

1. Existing architecture assessment
2. Existing database/schema assessment
3. What is already implemented
4. What is missing
5. What needs modification
6. Proposed Prisma relationship changes
7. Proposed API changes
8. How each of the **four flows** maps to the database
9. Potential conflicts with existing implementation

Then implement the changes.

Do not rewrite working parts unnecessarily.

59. AFTER IMPLEMENTATION

After implementation, provide a concise report containing:

Database Changes

What models/relations/migrations were added or changed.

Backend Changes

Which APIs/services were added or updated.

FastAPI / AI Changes

What AI-generation/extraction logic changed.

Frontend Integration

Which screens are now connected to real backend/database data.

Four Flow Verification

Explicitly confirm and describe:

- Flow 1 — General Assessment
- Flow 2 — Basic MCQ
- Flow 3 — Basic + Skills MCQ
- Flow 4 — Coding Assessment

Edge Cases Fixed

List the important failure conditions handled.

Remaining Issues

Do not claim something works if it has not actually been implemented or tested.

Final Instruction

This is not only a UI task.

Treat this as a complete **frontend** → **backend** → **database** → **AI service** → **backend** → **frontend** integration.

The most important requirement is that the system has **exactly four assessment flows** and that each flow works correctly from the first database query through assessment completion.

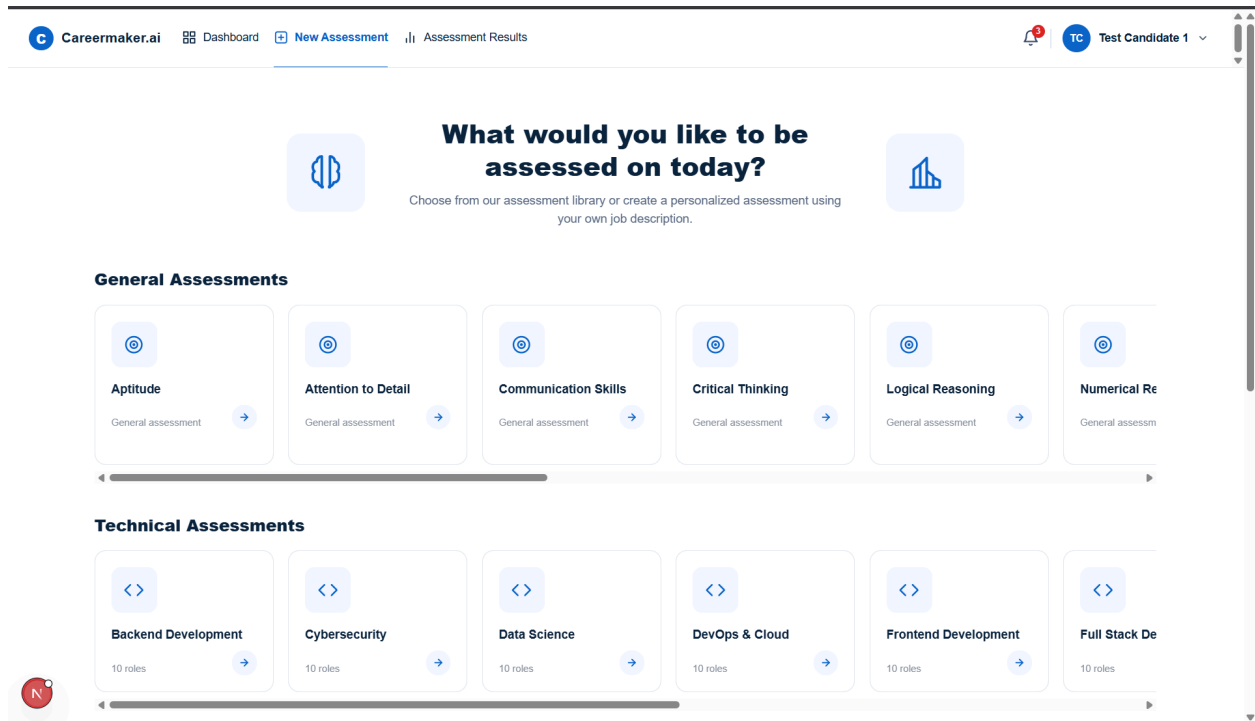
Before changing code, trace the complete data path.

After changing code, trace it again and verify that there are no broken transitions, missing persistence steps, incorrect relationships, duplicated questions, exposed answers, invalid assessment states, or frontend-only business rules.

Prefer correctness, maintainability, database integrity, and predictable behavior over shortcuts.

changes 3

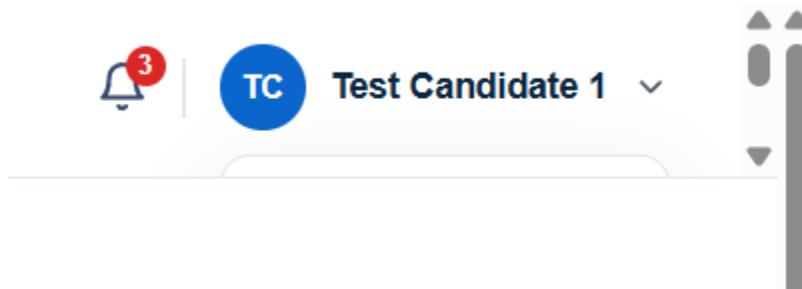
the new assment \



Firstly the dashboard and the new assessment and the assessment result selection to bar need to be centred properly and next to the notification button there should be a search button that allows user to search anything like category and assessments or area or whatever

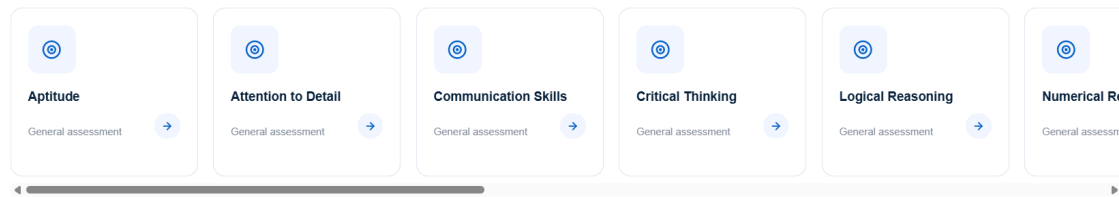
And the next thing is that the hero section ethics is the what would you like to be assessed on today card is doesn't look good and has no good animation

And then the general assessments there are the assessments



The top bar that has the user profile and name and drop down is having a scrollbar that is breaking this look and UI. If you use the scrollbar, the components like the profile and the tab selection almost everything up to nothing is visible. Please look into it and fix it.

General Assessments



Look at the assessments card there have icons that shouldn't be the case the admin will select the image to be displayed on those cards for now your task is to remove those icons and put in random images that would make it look like the reference

Also the scrolling mechanism should be like Netflix that is at one time the user for example is looking at the general assessment and sees 6 cards of assessments in full view that also need to be changed into 5 are in full view and one is getting cut off that needs to be fixed and all 6 need to be in full view there must be a subtle arrow on the right side after the 6th assessment card and as the user clicks on it the 6 cards before are slid to the left and new 6 cards show up this is how Netflix does this all the cards not only general assessments must behave like this

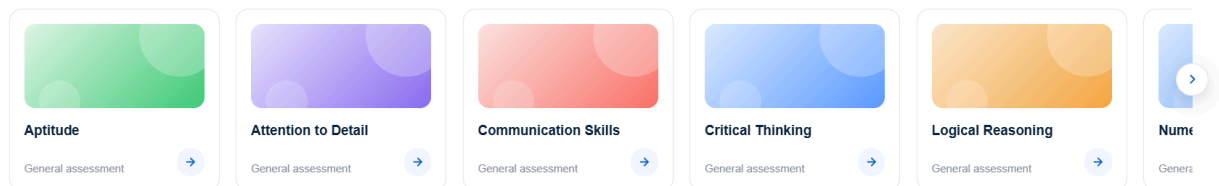
Tab 14



Firstly the logo is this : so make the logo
and the text look same to same that is on the top left cornter of the assessment ai

I have plans for the hero section in the new iassessmne t thelo i plan to use a video as either
heroes cards cback ground so your task is to prepare for it next ill youve claud e cod ethe viedo
and you are suposed to put it on

General Assessments

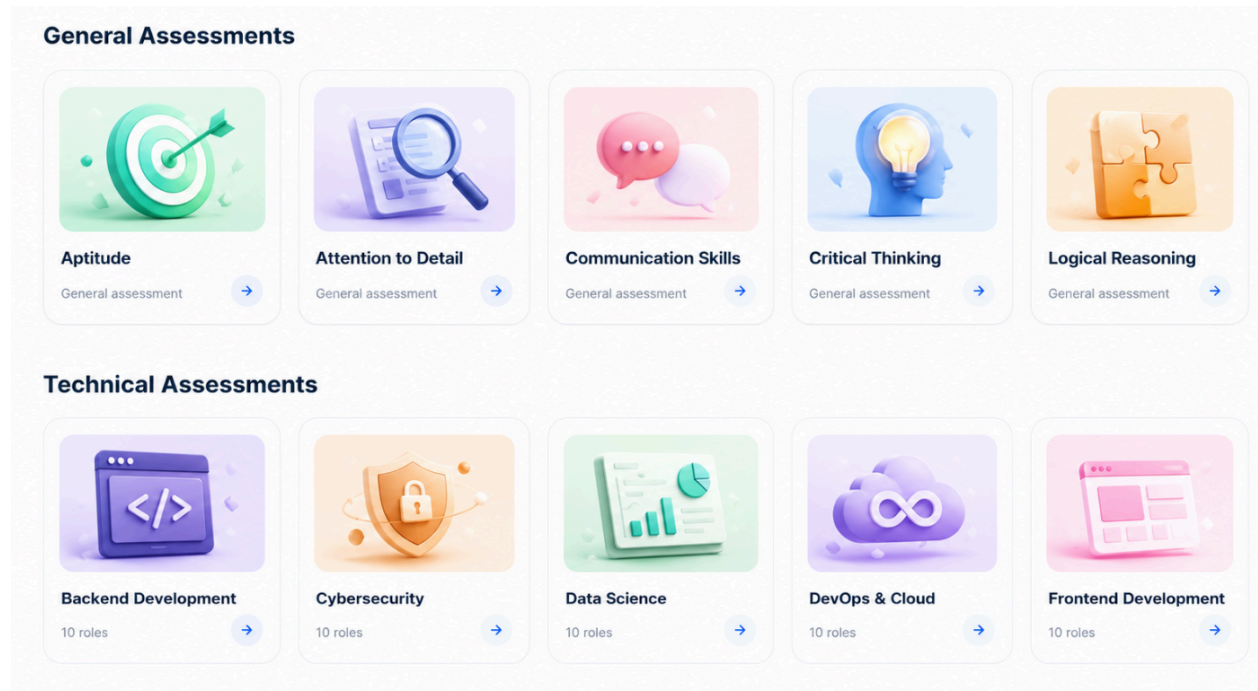


Still the general assessment s and the others are scrollable but the funcatanolituy il\of the
setflixstyle 5 cards sliding is there just disable the manual scroll only the buttons must work and
at a time show only 5 cards you can see in the image the cards on ther ight saying something
num is getting cut off i dont want that please make it only 5 fully visible

And all the category one after the other will be there technical assmnt i after htat what ever after
that what ever

And htne also the images in each assessment card

It schedule look somethin kiethis image below :



As you can see the image are nice and big and covering good part of it and the current design looks very rectangular like the y axis from my view is not long at all look at this proportions carefully and try copying the images as shown in this image and put them in randomly for all the cards accordingly

Ok now the filter thing that needs to be added now the user will be given the option to filter out and select which categories will be visible to the user when the user clicks the filter button on the right model corner after the hero ends for the hero its the right lower corner outside the filter button will show the categories and the user can select the categories he wants they can multi select when they select it becomes blue that means that is selected when clicked again to unselect the category option goes back to white here is a ai generated ref :

 Filter 



General Assessments



Technical Assessments



Business Assessments



IT & Software



Creative & Design



Sales & Marketing



Finance & Accounting



Healthcare



Education

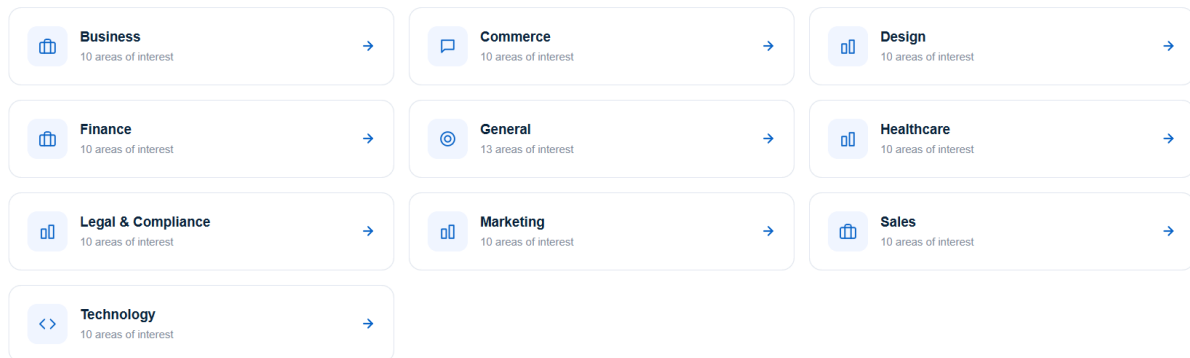


Operations

The browse by category:

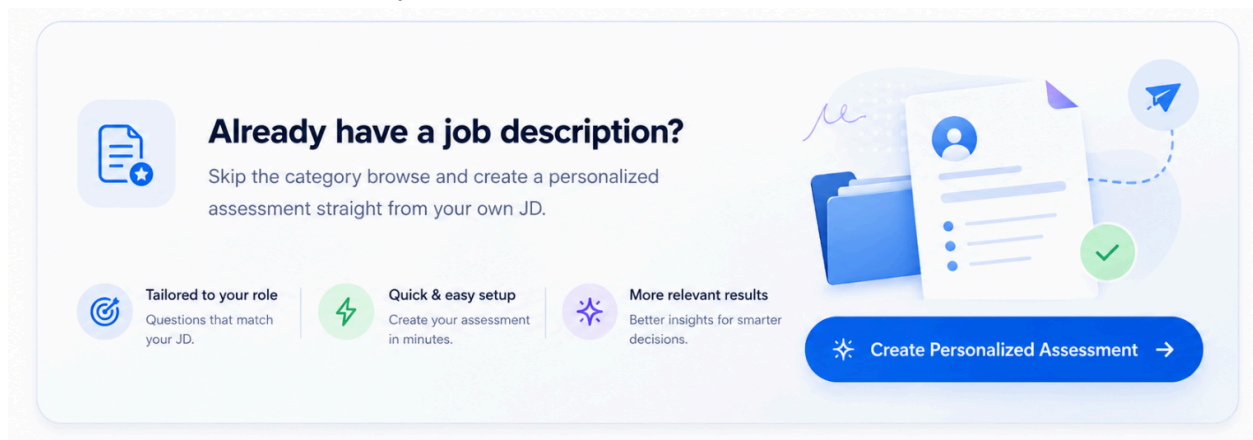
Browse by Category

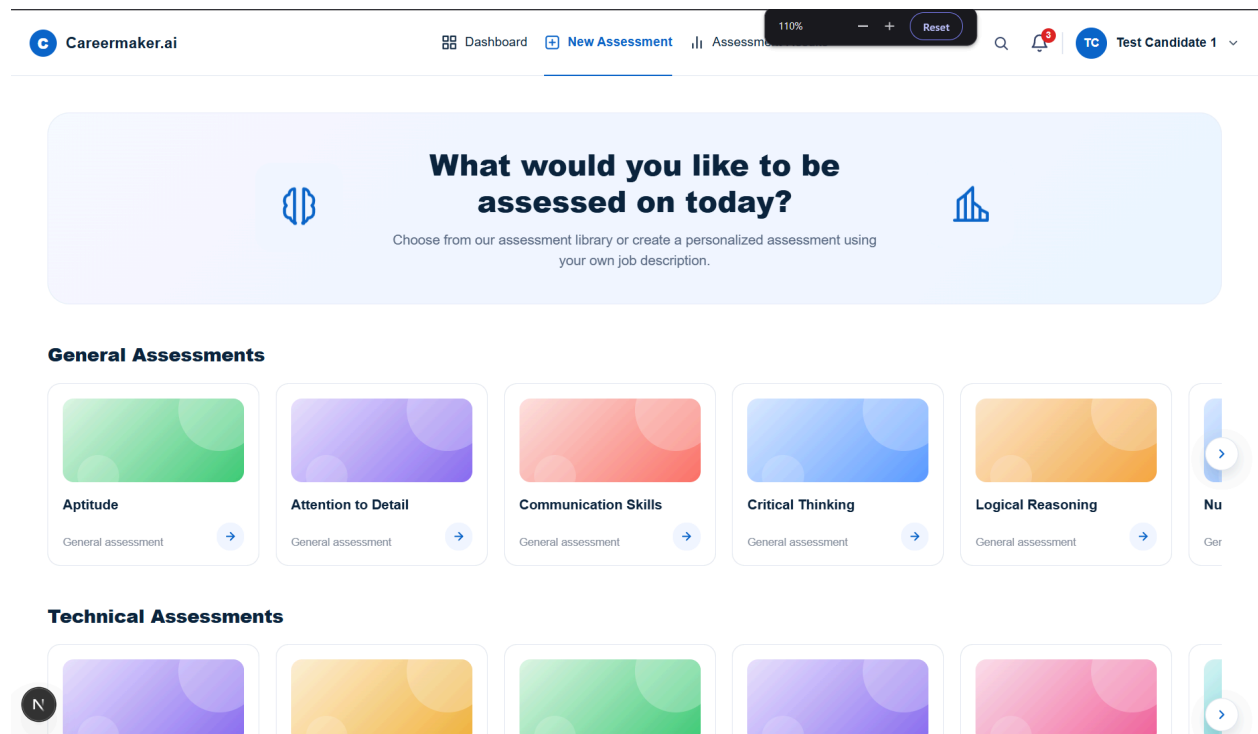
Every category opens into its Areas of Interest, then the specific role you want to be assessed for.



This section should only appear when the user selects only one category from the filter option if the user selects more than one this should not appear and the user will be only seeing the categories in the same Netflix 5 cards format

Change the look of the already have a job description ? cards that is used to create a personalized test to this exactly :





Look that the web iste when i put in in 110 zoom on my browser this look somuch better than befro because on the left and right ends of the screen less white unused space is ther so can you make change such that the website is zoomed in thai msuch ot gets wider or idk hwo to say it but i hope u usnderstand this looks better and i i want this look